

Python – Loops

אחיהוד

מה זה לולאה

לולאה נותנת לנו לחזור על אותו קוד באופן אוטומטי, מבלי לכתוב את אותן השורות שוב ושוב.

ב-Python יש שתי לולאות:

for לולאת

while לולאת

Exercise – "for" loops

1. עשו לולאת for על ה-list והדפיסו את כל השמות בתוכו:

```
colors= ["red", "blue", "green"]
```

2. צרו משתנה String, עשו עליו לולאה והדפיסו.

3. צרו משתנה dictionary עם שלושה key:value והדפיסו.

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python loops.py
M
y
n
a
m
e
```

```
names = "My name"
for i in names:
    print(i)
```

פתרון

1.

```
colors= ["red", "blue", "green"]
for i in colors :
    print(i)
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python loops.py
red
blue
green
```

2.

```
my_name = "Anna"
for i in my_name:
    print(i)
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python loops.py
A
n
n
a
```

Python – Loops

אחידוד

```
my_team = {"team_member1": "team_leader",  
           "team_member2": "Scrum_head",  
           "team_member3": "manager",  
           }  
for i in my_team:  
    print(i)
```

.3

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python loops.py  
team_member1  
team_member2  
team_member3
```

Python

לולאת For

אחיהוד

מה זה לולאת for

לולאה שיודעת לרוץ על שורות של קוד ששמנו אותן בלולאה.

לולאת **for** יודעת לרוץ על ערכים בתוך משתנים שהם containers (list, tuple).

הלולאה תסתיים רק אחרי שהיא תעבור **על כל הערכים** בתוך המשתנה.

לולאת **for** תמיד תבוא עם משתנה (למשל: "i") שירוצן על כל הערכים + פקודות **in** שמכוונות על איזה משתנה אנחנו רוצים לעשות לולאה .

```
names = ["Alice", "Bob", "Charlie"]
for i in names:
    print(i)
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals>
```

```
python loops.py
```

```
Alice
Bob
Charlie
```

Python

לולאת For

למה משתמשים ב-i

אחידות

מפתחים אוהבים להשתמש במשתנה i

מפתחים כבר התרגלו לקרוא למשתנה שרץ על לולאה : i

יש כאלה שגם משתמשים ב-j זה בא מעולם המתמטיקה.

יש שאומרים, i זה מהמילה iterations (חזרות).

ה- i הוא בסך הכל "קופסה" זמנית שמחזיקה מידע

בכל סיבוב של הלולאה הוא פשוט מכיל את המילה הנוכחית מתוך הרשימה.

הוא לא עושה שום פעולה, הוא רק מחזיק את הנתון כדי שאנחנו נוכל לעשות עליו פעולות בתוך גוף הלולאה.

1. במידה ורוצים לחפש בעזרת האות i, האות i היא לא פונקציה ולא חיפוש, האות i היא פשוט המילה עצמה.
את החיפוש נעשה בתוך הלולאה, על גבי ה-i.
2. ה- i לעולם לא "ריק" בזמן שהלולאה רצה.
בכל סיבוב של הלולאה, ה- i מלא בערך אחר מתוך הרשימה.
ה- i מקבל משמעות ברגע שהלולאה מתחילה לרוץ.

הסבר

אם יש לנו רשימה של מילים ואנחנו רוצים למצוא באיזה מהן יש את האות 'e', אנחנו מפרידים בין המידע (ה- i) לבין הפעולה (החיפוש).

```
words = ["pink", "magenta", "rose", "fuchsia"]  
  
for i in words:  
    if "e" in i:  
        print(i)
```

לדוגמא

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python loops.py  
magenta  
rose
```

Python

לולאת For

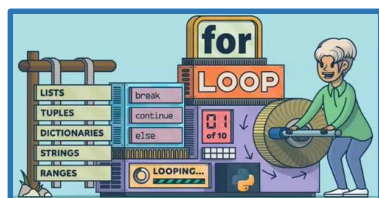
למה משתמשים ב-*i*

אחידות

1. ה- *i* היא לא הפונקציה.

ה- *i* היא פשוט המילה התורנית (פעם אחת "pink" פעם שנייה "magenta" וכו')

2. הפעולה של החיפוש מתבצעת בשורה הבאה (if "e" in i), שבה אנחנו שואלים את פייתון, האם האות e נמצאת בתוך המילה שעכשיו שמורה בקופסה *i*



Python

לולאת For

אחיהוד

חשוב להבין על לולאת for

לולאת **for** יכולה לרוץ רק על Data Types שבאים בסוג של container, למשל:

List/tuple – אפשר מכיוון שהם **containers** שמכילים בתוכם ערך או כמה ערכים .

String – אפשר אבל, משתנה "i" ירוץ על כל אות במילה (המילה היא ה-container).

Dict – אפשר אבל, משתנה "i" ירוץ רק על ה-keys .

Int/float – אי אפשר, הם לא **containers** (רק אם הם באים בתוך list/dict).

Boolean – אי אפשר, הוא לא **container** (רק אם הוא יבוא בתוך list/dict).

Python

לולאת For

Syntax

אחידוד

1. משתנה "names" סוג list .
2. לולאת **for** .
3. משתנה ריק בשם "i" שיקח בתוכו ערך ערך .
4. הפקודה "in" מכוונת לשם של המשתנה "names" .
5. בסוף הפקודה לשם נקודתיים ואז מה שאנחנו רוצים, שיקרה.

*כמה פעמים הלולאה תרוץ על כל שורות הקוד ?

```
names = ["Alice", "Bob", "Charlie"]
for i in names:
    print(i)
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals>
```

```
python loops.py
```

```
Alice
Bob
Charlie
```

Python

לולאת For

אחידוד

פייתון מדפיס אחד אחד .

ב-VS Code, אנחנו רואים שהוא מדפיס לנו את שלושתם –בבת אחת.

אומנם, זה לא המקרה.

הוא מדפיס אחד כל כך מהר, כך שזה נראה לנו כאילו הוא מדפיס אותם ביחד.

```
script
a_list = [1, 3, 5]

for value in a_list:
    print(value) ←
```

First Iteration

Output

```
1 ←
```

```
script
a_list = [1, 3, 5]

for value in a_list:
    print(value) ←
```

First Second Iteration

Output

```
1
3 ←
```

```
script
a_list = [1, 3, 5]

for value in a_list:
    print(value) ←
```

First Second Third Iteration

Output

```
1
3
5 ←
```

Python



לולאת *if/elif/else*

+

Conditions

אחידוד

לולאת for עם תנאי if/elif/else

ניתן לשלב `if` בתוך הלולאה .

ה-`if` גם הוא יכנס לתוך הלולאת `for` וירון כמספר הפעמים שהלולאה רצה (כמספר הערכים בתוך ה-container)

להבין

הלולאה : חוזרת על פעולות.

התנאי : מקבל החלטות.

לדוגמא

מה הקוד הזה עושה ?

```
names = ["Alice", "Alex", "Anton", "Maria"]
for i in names:
    if i == "Maria":
        print("Maria Found")
    else:
        print("Maria not Found")
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python loops.py
Maria not Found
Maria not Found
Maria not Found
Maria Found
```

Exercise – for loop + if->elif->else

1. נתון ה-list `numbers = [3, -1, 0, 5, -7]` :

כתבו קוד שעובר על כל הערכים ובודק אם המספר :

1. חיובי (הדפיסו "Positive")

2. שווה לאפס (הדפיסו "Zero")

Python



לולאת *if/elif/else*

+

Conditions

אחידות

3. שלילי (הדפיסו "Negative")

פתרון

```
numbers = [3, -1, 0, 5, -7]

for i in numbers:
    if i > 0:
        print("Positive")
    elif i == 0:
        print("Zero")
    elif i < 0:
        print("Negative")
    else:
        print("undefined")
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python loops.py
Positive
Negative
Zero
Positive
Negative
```

Exercise – for loop + if->elif->else

2. נתון ה-list: `ages = [10, 15, 30, 18]`

כתבו קוד שיעובר על כל הערכים ובודק אם המספר :

```
ages = [10, 15, 30, 18]

for i in ages:
    if i < 13:
        print("Child")
    elif i > 18:
        print("Adult")
    else:
        print("Teen")
```

1. קטן מ-13 (הדפיסו "Child") : פתרון
2. בין 13-18 (הדפיסו "Teen")
3. גדול מ-18 (הדפיסו "Adult") .

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python loops.py
Child
Teen
Adult
Teen
```

Python

לולאת While

אחיהוד

מה זה לולאת while

לולאה שחוזרת על אותן שורות קוד, כל עוד התנאי נכון .

בשונה מ-for, לולאת while לא צריכה container .

בלולאת while, התנאי הוא מה שקובע.

משתנה $i = 0$

זה התנאי : כל עוד i קטן מחמש – הדפס את i .

בנוסף, כל פעם שתגיע לסוף הלולאה, i יהיה שווה $1 + i$.

```
i = 0
while i < 5 :
    print(i)
    i = i + 1
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python loops.py
0
1
2
3
4
```

חוק ברזל בלולאת while

כל לולאת while חייבת פקודה סופית שתגרום לה להפסיק.

אנחנו רואים שבסוף הפקודה, i תמיד גודל ב-2 .

הפקודה הזאת נועדה לגרום לתנאי שבלולאה, בסופו של דבר, להיות לא נכונה.

כלומר, בחזרה החמישית, התנאי יהיה לא נכון והלולאה תסתיים.

```
i = 2
while i < 10 :
    print(i)
    i = i + 2
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python loops.py
2
4
6
8
```

Python

לולאת While

אחיהוד

ההבדל בין הלולאות

for לולאת

תעבור על כל ערך ב-container ותסיים עצמאית.

while לולאת

תמשיך לרוץ כל עוד התנאי נכון (תעצור כשהתנאי יפסיק להיות נכון)

Exercise - "while" loop

דוגמא

```
i = 0
while i < 5 :
    print(i)
    i = i + 1
```

1. הדפיסו מספרים מ-20 ל-25
2. הדפיסו מספרים מ-5 ל-1
3. הדפיסו "Hello" 4 פעמים
4. הדפיסו מספרים זוגיים מ-1 עד 10 .

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python loops.py
0
1
2
3
4
```

פתרון

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python loops.py
20
21
22
23
24
25
```

```
i = 20
while i < 26 :
    print(i)
    i = i + 1
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python loops.py
2
4
6
8
10
```

```
i = 2
while i < 12 :
    print(i)
    i = i + 2
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python loops.py
5
4
3
2
1
```

```
i = 5
while i > 0 :
    print(i)
    i = i - 1
```

```
PS C:\Users\LENOVO\Desktop\JS_Fundamentals> python loops.py
Hello
Hello
Hello
Hello
```

```
i = 1
while i < 5 :
    print("Hello")
    i = i + 1
```

Python

לולאת While

אחיהוד

הסבר לסעיף 2 (הדפסת מספרים מ-5 ל-1)

עד עכשיו, בסעיפים 1 ו-4, הלול אה "טיפסה במדרגות", ממספר נמוך, הגדרנו ללולאה תנאי של "תרוצי כל עוד המספר קטן מהתקרה שלי", ובכל סיבוב של הלולאה הוספנו למספר עוד קצת (צעדת למעלה).

בסעיף 2 אנחנו עושים בדיוק את ההיפך: אנחנו צריכים לרדת במדרגות .

בפיתוח אוטומציה, חושבים על מצב שבו יש טסט שבודק התחברות למערכת, ורוצים לתת לו 5 ניסיונות חוזרים למקרה שהשרת איטי.

מתחילים לספור לאחור: 5, 4, 3, 2, 1... וכשאין יותר ניסיונות - הטסט נכשל.

איך לולאת while יורדת

נקודת ההתחלה, צריכה להיות הפעם גבוהה ולא נמוכה.

התנאי

במקום לשאול "האם אני עדיין מתחת לתקרה?", הלולאה צריכה לשאול "האם אני עדיין מעל הרצפה?".

כלומר, צריכים לשנות את החשיבה של סימן הגדול/קטן, כי הפעם המספרים הולכים ויורדים, ורוצים שהלולאה תרוץ כל עוד לא "התרסקת" אל מתחת ליעד הסופי.

ההתקדמות

בסוף הלולאה, במקום לחבר ולהוסיף צעדים, צריכים "להחסיר" מדרגה אחת כדי להתקרב לרצפה.

הסבר לסעיף 3 שימוש בלולאת while, כמונה לביצוע פעולה קבועה

כאשר אנחנו מעוניינים לבצע פעולה זהה מספר מוגדר של פעמים, נשתמש במשתנה הלולאה אך ורק לצורך ספירת הפעמים (איטרציות), ולא כחלק מהפלט עצמו.

אתחול המונה

נגדיר משתנה עזר למשל $i = 1$, המשתנה מייצג את הניסיון או ה"סיבוב" הראשון של הלולאה.

Python

לולאת While

אחיהוד

תנאי הלולאה

נגדיר את תנאי העצירה בשורת ה- while בהתאם למספר הפעמים שאנו רוצים שהפעולה תרוץ, למשל כדי שהלולאה תרוץ 4 פעמים, נגדיר שהיא תעבוד כל עוד

$i < 5$ (הסיבוב החמישי כבר לא יתבצע)

הפעולה בגוף הלולאה

בתוך הפקודה `print()`, נעביר את הערך הקבוע שאנחנו רוצים להדפיס כטקסט (מחרוזת בתוך מרכאות למשל "Hello") ולא את המשתנה `i`.

מניעת לולאה אינסופית

השלב הקריטי בסוף כל איטרציה הוא קידום המונה על ידי הוספת 1 ($i = i + 1$)

פעולה זו מבטיחה שהמונה יתקדם בכל סיבוב, עד שיחרוג מהתנאי שהגדרנו והלולאה תעצור באופן תקין.

Python

מתי נשתמש ב-*for* או *while*

אחידות

For	While	
רוץ על כל אחד מ....	רוץ כל עוד	
כשמספר החזרות בלולאה ידוע לנו	כשמספר החזרות בלולאה תלוי בתנאי שהצבנו	מתי ?
עובד רק בתוך Data Type container	לא נצטרך container	איך נשתמש ?
תרוץ על כל הערכים בתוך המשתנה (container) ותעצור עצמאית בסיום.	תמשיך לרוץ כל עוד התנאי נכון . *להיזהר מלולאה אינסופית	חשוב שנוזכר
<pre>names = "My name" for i in names: print(i)</pre>	<pre>i = 2 while i <= 10 : print(i) i = i + 1</pre>	Syntax

Python

לולאת While

דוגמאות לא טובות של לולאות while

אחידוד

```
i = 2
while i < 10:
    print (i)
```

i אף פעם לא משתנה .

לולאה אינסופית.

הסבר

ה-i שווה ל-2 ,

2 קטן מ-10 ,

הדפס i (2),

הכל פה נכון, איך נצא מהלולאה, ידפיס 2 בלי סוף !

```
i = 2
while i > 10:
    print (i)
    i = i + 2
```

תנאי לא נכון .

לא ידפיס כלום

(2 לא יותר גדול מ-10)

הסבר

ה-i שווה ל-2 ,

2 גדול מ-10 ,

אבל 2 לא גדול מ-10, אז התנאי לא נכון (לא ידפיס כלום).

ההמשך של הלולאה כבר לא משנה, כי היא נעצרת בתנאי הלא נכון ולא ממשיכה .



Python

While לולאת

אחיהוד

זהירות !

לולאת while יכולה לרוץ לנצח .

Ctrl+c כשהסמן בתוך הטרמינל או לצאת מהתוכנה כדי להפסיק .